

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Kurt Wiederhold

11 de septiembre de 2026

20:23

1. Introducción

Este trabajo estudia experimentalmente dos problemas clásicos: ordenar un arreglo de enteros y multiplicar dos matrices cuadradas. Para el primero se comparan `std::sort`, merge sort, quick sort y patience sort [6, 1]; para el segundo, el método clásico de tres bucles frente al algoritmo de Strassen [9]. Sus complejidades teóricas son conocidas, de modo que el objetivo no es deducirlas sino contrastarlas con el comportamiento observado sobre los casos del anexo A del enunciado, variando el orden inicial de la entrada, la estructura de la matriz y el dominio de los valores. La pregunta que guía el análisis es qué esconde la notación asintótica: dos algoritmos del mismo orden pueden diferir en un factor constante grande, un exponente medido puede no coincidir con el teórico, y un algoritmo asintóticamente superior puede perder en el rango de tamaños que interesa en la práctica.

2. Entorno experimental

Plataforma. AMD Ryzen 7 7700 (8 núcleos Zen 4, 3,8 GHz), 32 GiB DDR5, cachés de 32 KiB (L1d, 8 vías), 1 MiB (L2) por núcleo y 32 MiB (L3); Windows 11 Pro 24H2; g++ 16.1.0 (MinGW-w64), opciones `-std=c++17` y `-O2`, sin `-march=native` para no atar el binario a este procesador [4]; Python 3.12.10, NumPy 2.5.2 y Matplotlib 3.11.1.

Datos. Exactamente los del anexo A: 72 arreglos y 72 pares de matrices (4 tamaños $\times$ 3 tipos $\times$ 2 dominios $\times$ 3 muestras), producidos sin modificación por los generadores entregados (`array_generator.py`, `matrix_generator.py`).

Metodología. Se cronometra sólo la llamada al algoritmo, con `steady_clock`; la E/S y la verificación quedan fuera porque para $n = 10^7$ leer el archivo tarda más que ordenarlo. Cada caso se repite 15, 7 o 3 veces según su tamaño; se reporta la *mediana*, robusta frente a los valores atípicos altos del planificador [7]. Cuando una ejecución dura menos que la resolución del reloj (≈ 100 ns) se cronometra un lote preparado de antemano y se divide el total. La memoria auxiliar se mide reemplazando los operadores globales `new/delete` por versiones instrumentadas. Toda salida se verifica contra una referencia independiente: las 2 880 mediciones de ordenamiento, las 1 440 de matrices, las 11 988 del barrido de umbral y las 18+54 de los dos controles resultaron correctas. Figuras, tablas y toda cifra citada en el texto se generan desde los CSV con los scripts `plot_generator.py`.

3. Resultados y Análisis

Figuras y tablas se generan desde los CSV con `plot_generator.py` y quedan en `data/plots/` de `code/sorting` y de `code/matrix_multiplication`.

3.1. Ordenamiento de arreglos

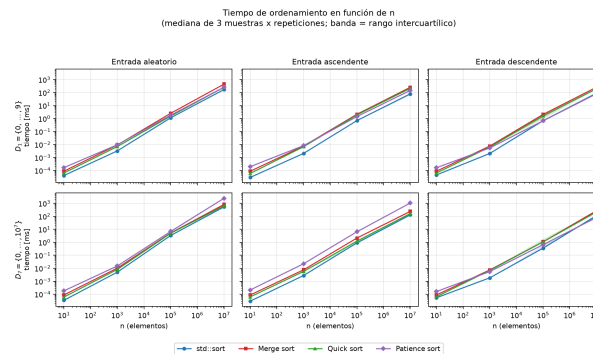


Figura 1: Tiempo frente a n (doble logarítmica), por orden inicial (columnas) y dominio (filas). Las curvas son casi paralelas: lo que separa a los algoritmos es el factor constante, no el orden de crecimiento.

Cuadro 1: Tiempo mediano [ms], entrada *aleatoria* con dominio D_7 (mediana sobre las tres muestras y todas las repeticiones).

Algoritmo	$n = 10^1$	$n = 10^3$	$n = 10^5$	$n = 10^7$
std::sort	$3,44 \times 10^{-5}$	$5,05 \times 10^{-3}$	3,37	535
Merge sort	$8,90 \times 10^{-5}$	0,0104	5,21	829
Quick sort	$6,16 \times 10^{-5}$	$8,19 \times 10^{-3}$	4,98	653
Patience sort	$1,84 \times 10^{-4}$	0,0151	6,90	2469

En escala doble logarítmica una ley de potencias es una recta, y en la Fig. 1 las cuatro curvas son rectas casi paralelas: ningún algoritmo cambia de orden. El ajuste $t = c n^p$ de la Tabla 2 lo confirma, con

exponentes entre 0,96 y 1,22 en torno al 1,14 de $n \log_2 n$ en este rango. La notación asintótica acierta, y es todo lo que acierta: no distingue entre las cuatro curvas, y esas diferencias son grandes.

Las constantes ocultas. En la [Tabla 1](#), para $n = 10^7$ con entrada aleatoria, quick sort tarda 1,22 veces lo que `std::sort`, merge sort 1,55 y patience sort 4,61. Los cuatro son $\Theta(n \log n)$ y aun así el peor cuesta $4,6 \times$ el mejor. La ventaja de `std::sort` es de ingeniería, no de orden: introsort remata los subarreglos de tamaño ≤ 16 con una única pasada de inserción sobre todo el arreglo [8, 2].

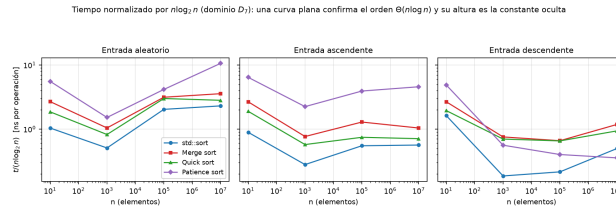


Figura 2: Tiempo dividido por $n \log_2 n$ (ns por operación). Con costo uniforme por operación la curva sería horizontal, a la altura de la constante oculta.

El costo por operación no es constante. Las curvas de la [Fig. 2](#) no son horizontales: entre $n = 10^3$ y $n = 10^5$ el costo por operación de `std::sort` sobre entrada aleatoria salta de 0,51 a 2,03 ns, un factor 4,0. La interpretación más plausible es la jerarquía de memoria: 10^3 enteros ocupan 4 KiB y caben en L1 (32 KiB); 10^5 ocupan 400 KiB y sólo caben en L2 (1 MiB); 10^7 ocupan 40 MiB y ya no caben en L3 (32 MiB), y un fallo de L3 cuesta unas dos órdenes de magnitud más que un acierto de L1 [3] (interpretación: aquí no se miden fallos de caché). A partir de $n = 10^5$ la curva se aplanan y ahí sí se lee la constante de cada algoritmo.

La forma de la entrada importa, incluso donde la teoría dice que no. Con $n = 10^7$ y dominio D_7 , pasar de la entrada aleatoria a la más favorable de las dos ordenadas divide el tiempo por 4,6 en `std::sort`, 4,0 en quick sort y 3,4 en merge sort. Este último es el caso interesante: es deliberadamente insensible a los datos (sin el atajo que salta la mezcla de mitades ya ordenadas) y mueve los mismos elementos con cualquier permutación. Dos efectos que la cuenta de operaciones no ve pueden explicarlo: con entrada ordenada cada mezcla agota primero el tramo izquierdo (exactamente la mitad de las comparaciones) y $a[i] \leq a[j]$ es siempre verdadera, lo que favorecería al predictor de saltos (hipótesis no medida). El dominio produce un efecto análogo: con n fijo y entrada aleatoria, pasar de D_1 a D_7 multiplica el tiempo por 3,1 en `std::sort` y por 9,4 en patience sort, con el mismo n .

Patience sort: el peor caso medido no es el peor caso teórico. Con entrada descendente todo cae en una sola pila, la mezcla es trivial y el algoritmo es $\Theta(n)$: tarda 82,5 ms y gana a los 115,2 ms de `std::sort`. La teoría señala como peor caso la entrada ascendente, que abre una pila por valor distinto ($k \approx 6,3$ millones en D_7 ; los repetidos comparten pila) y sube a 1063 ms. Pero el peor caso *medido* es la entrada aleatoria: 2469 ms, 2,3 veces la ascendente, pese a generar sólo $k \approx 2\sqrt{n} \approx 6325$ pilas. La hipótesis más plausible es de nuevo la memoria: con pocas pilas largas, la mezcla sigue el puntero `below[i]` hacia una posición *aleatoria* de dos arreglos de 40 MiB (dos accesos sin localidad por elemento), mientras que con la entrada ascendente las pilas tienen uno o dos elementos y la búsqueda binaria recorre siempre el

mismo camino. La razón entre su mejor y su peor caso medidos es 29,9, frente a 3,4–4,6 de los otros tres.

Cuadro 2: Exponente empírico p de $t = cn^p$ (mínimos cuadrados en log-log, dominio D_7). Un $\Theta(n \log n)$ debe dar $p \approx 1,14$, la pendiente de $n \log_2 n$ en el rango medido. Entre paréntesis, R^2 .

Algoritmo	aleatorio	ascendente	descendente
std::sort	1,220 (0,997)	1,123 (0,998)	1,064 (0,989)
Merge sort	1,180 (0,998)	1,088 (0,998)	1,082 (0,998)
Quick sort	1,193 (0,997)	1,079 (0,999)	1,089 (0,999)
Patience sort	1,202 (0,995)	1,128 (0,999)	0,960 (0,995)

Memoria. La firma del `sort.cpp` entregado devuelve una copia del arreglo, así que toda llamada asigna al menos $4n$ bytes; con ese piso, la [Tabla 3](#) separa los dos grupos que predice la teoría. `std::sort` y `quick sort` dan exactamente $4n$: sólo la copia, sin memoria auxiliar propia ($\Theta(1)$ más $\Theta(\log n)$ de pila, que este instrumento no ve). `Merge sort` da $8n$: la copia más su búfer. `Patience sort` llega a $8n$ en el caso típico y a 156,5 MiB (4,1 veces el arreglo) en el ascendente D_7 , por los $\approx 6,3$ millones de entradas del heap de mezcla: paga con memoria su ventaja en el caso descendente.

Cuadro 3: Pico de memoria dinámica de la llamada (máximo sobre órdenes, dominios y muestras), incluida la copia de retorno de $4n$ bytes de la firma provista: un valor de exactamente $4n$ significa sin memoria auxiliar propia.

Algoritmo	$n = 10^1$	$n = 10^3$	$n = 10^5$	$n = 10^7$
std::sort	40 B	3,9 KiB	390,6 KiB	38,1 MiB
Merge sort	80 B	7,8 KiB	781,2 KiB	76,3 MiB
Quick sort	40 B	3,9 KiB	390,6 KiB	38,1 MiB
Patience sort	416 B	19,6 KiB	2,0 MiB	156,5 MiB

3.2. Multiplicación de matrices

Cuadro 4: Tiempo mediano [ms], matrices *densas* con dominio D_{10} , umbral de corte de Strassen 64; última fila: cociente.

Algoritmo	$n = 2^4$	$n = 2^6$	$n = 2^8$	$n = 2^{10}$
Clásico (naive)	$1,23 \times 10^{-3}$	0,0985	16,87	2630
Strassen	$1,24 \times 10^{-3}$	0,0981	6,62	464
Clásico / Strassen	1,00	1,00	2,55	5,67

Strassen gana, pero sólo cuando n es grande. En la [Tabla 4](#) ambos algoritmos son indistinguibles para $n = 2^4$ y $n = 2^6$ —con el umbral en 64 la recursión no llega a activarse y Strassen es el método clásico— y luego se separan: 2,55 veces más rápido en $n = 2^8$ y 5,67 veces en $n = 2^{10}$ (464 frente a 2630 ms). El punto



Figura 3: Izquierda: tiempo frente a n , matrices densas, con las cotas teóricas ancladas al último punto del clásico. Derecha: efecto del umbral de corte de Strassen (mediana sobre tipos y dominios).

de cruce está entre 2^6 y 2^8 y la ventaja crece con n , como corresponde a una diferencia de exponente. Se paga en memoria: para $n = 2^{10}$ el clásico usa 8,0 MiB —exactamente la matriz resultado, $8n^2$ bytes— y Strassen 31,9 MiB, 4,0 veces más, por los siete productos y dos acumuladores de cada nivel.

Cuadro 5: Control (make stride-check): tiempo y costo por operación del clásico, matrices densas D_{10} , en $n = 2^k$ y en dimensiones vecinas de igual memoria; sólo cambia el alineamiento del salto de $8n$ bytes.

n	250	256	260	500	512	520	1000	1024	1040
tiempo [ms]	5,06	16,66	5,53	58,66	363	69,48	497	3174	533
t/n^3 [ns]	0,32	0,99	0,31	0,47	2,70	0,49	0,50	2,96	0,47

Los exponentes medidos superan a los teóricos, y la causa no es la que parece. El ajuste $t = c n^p$ sobre las matrices densas da $p = 3,525$ para el clásico (teoría: 3; $R^2 \geq 0,998$) y $p = 3,066$ para Strassen entre los 2 tamaños en que su recursión está activa (teoría: $\log_2 7 = 2,807$). La explicación inmediata —«la matriz deja de caber en caché»— queda descartada por el control de la [Tabla 5](#): matrices de 1000 y 1040 ocupan la misma memoria que una de 1024 y tampoco caben en L3, pero el clásico las multiplica a 0,47–0,50 ns por operación, mientras que en 1024 cuesta 2,96 ns: 6,1 veces más. Lo que cambia entre esos tamaños es sólo el *stride*: el bucle interno recorre B por columnas y, cuando n es potencia de dos, el salto de $8n$ bytes es múltiplo del tamaño de página. El mecanismo estándar para ese patrón [3] —todos los elementos de una columna caen en el mismo conjunto de la caché (L1d: 64 conjuntos de 64 B) y, con 8 vías, cada acceso falla aunque la matriz quepa— es consistente con lo medido, aunque aquí no se contaron fallos de caché. Entre tamaños no potencia de dos, t/n^3 varía sólo $1,6\times$ de 250 a 1040 (ese sí sería el efecto de capacidad), frente a $3,0\times$ de 256 a 1024. Strassen sufre menos, presumiblemente porque sus operandos temporales son contiguos, pero sus hojas también leen bloques de B con stride $8n$; de ahí su $p > \log_2 7$. La consecuencia es importante: al costo por operación de sus vecinos, el clásico tardaría ≈ 521 ms en $n = 2^{10}$ y la ventaja de Strassen sería de $1,12\times$, no de $5,67\times$: buena parte de su «triumfo» es el castigo de la caché al clásico en las potencias de dos.

Sin umbral de corte, Strassen no sirve (curva en U del panel derecho de la [Fig. 3](#)). Para $n = 2^8$ (densa D_{10}), la recursión pura hasta bloques de 1×1 tarda 100,6 ms: 6,0 veces más que el clásico. El óptimo medido está en el umbral 16, con 4,28 ms (3,94 veces más rápido que el clásico); el umbral 64 usado en

el resto del experimento da 6,64 ms, razonable pero no óptimo: este parámetro se mide, no se supone. Las 18 sumas de bloques y la asignación de memoria por nivel son $\Theta(n^2)$ con constante alta y en bloques pequeños dominan el ahorro de $8 \rightarrow 7$ multiplicaciones [5].

La estructura de la matriz no importa; el estado de la máquina, sí. Como ninguno de los dos algoritmos inspecciona los coeficientes, una matriz diagonal debe costar lo mismo que una densa, y así ocurre: para $n = 2^{10}$ el clásico varía un 6% entre los seis pares tipo/dominio de la corrida principal, y el mismo 6% se obtiene al medir cada tipo en un proceso independiente y siempre en primera posición (type_order_check.py: 3 279, 3 359 y 3 168 ms), lo que descarta que el orden de las mediciones explique la diferencia [7]. El control mostró además algo que no se buscaba: sus tres tiempos son un 25% más altos que los mismos casos de la corrida principal. Afecta por igual a los tres tipos y no altera la conclusión; su causa no se determinó, y por eso este informe sólo compara mediciones de una misma corrida.

4. Conclusiones

La notación asintótica predijo la forma de todas las curvas y dejó fuera tres cosas. Las constantes: los cuatro ordenamientos son $\Theta(n \log n)$ y aun así patience sort tarda 4,61 veces lo que `std::sort` para $n = 10^7$. El costo uniforme por operación: el de `std::sort` se multiplica por 4,0 al dejar de caber en L1, y el clásico de matrices exhibe $p = 3,525$ en vez de 3 no por falta de caché sino por el alineamiento del recorrido por columnas en $n = 2^k$ ($n = 1000$ cuesta 6,1 veces menos por operación que 1024 con igual memoria). Y la forma de la entrada: el orden inicial altera hasta 29,9 veces el tiempo de patience sort, cuyo peor caso medido (aleatorio) no es el teórico (ascendente). Strassen resume la lección: su exponente es real ($5,67 \times$ en $n = 2^{10}$), pero sólo si la recursión se detiene a tiempo —hasta bloques 1×1 es 6,0 veces más lento que el $\Theta(n^3)$ — y frente a un clásico sin la penalización de las potencias de dos su ventaja sería de $1,12 \times$.

Código fuente. Todo el código, los scripts y las mediciones de esta tarea están en https://github.com/kurt-wiederhold/Tarea1_algoco_KurtWiederhold.

Referencias

- [1] Thomas H. Cormen et al. *Introduction to Algorithms*. 3.^a ed. Cambridge, MA: The MIT Press, 2009. ISBN: 9780262033848.
- [2] cppreference.com. *std::sort*. <https://en.cppreference.com/w/cpp/algorithm/sort>. 2024.
- [3] Ulrich Drepper. *What Every Programmer Should Know About Memory*. <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>. 2007.
- [4] David Hales. «An Introduction to Replicability in Computer Science Research». En: *Journal of Computational Methods* 13.1 (2017), págs. 123-145.

- [5] Steven Huss-Lederman et al. «Implementation of Strassen's Algorithm for Matrix Multiplication». En: *Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*. 1996. DOI: [10.1145/369028.369096](https://doi.org/10.1145/369028.369096).
- [6] Donald E. Knuth. *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN: 0201896850.
- [7] Catherine C. McGeoch. *A Guide to Experimental Algorithmics*. Cambridge: Cambridge University Press, 2012. ISBN: 9780521173018.
- [8] David R. Musser. «Introspective Sorting and Selection Algorithms». En: *Software: Practice and Experience* 27.8 (1997), págs. 983-993. DOI: [10.1002/\(SICI\)1097-024X\(199708\)27:8<983::AID-SPE117>3.0.CO;2-1](https://doi.org/10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-1).
- [9] Volker Strassen. «Gaussian elimination is not optimal». En: *Numerische Mathematik* 13.4 (1969), págs. 354-356. DOI: [10.1007/BF02165411](https://doi.org/10.1007/BF02165411).